SDS - REQ-IF-001 메일 발송 인터페이스

작성일 2026-10-17 · 근거: 요구사항정의서_엑셀양식_v3_1_3.xlsx (F열) · SRS 없음 · 대상: D:₩legacy-portal

1. 개요

요구사항 ID	REQ-IF-001
요구사항명	메일 발송 인터페이스
대 / 중 / 소분류	PORTAL / Common / Mail
우선순위	Medium
진행현황	테스트중 (비고: 소스 역추출.)
요약	결재(상신·승인·반려)·긴급공지 알림 메일을 `MailSender` 인터페이스 뒤에서 발송한다. 제목·본문 양식은 `MailFormat` 한 곳에서 만들고, 현재 구현체 `ConsoleMailSender` 가 콘솔에 `=== MAIL ===` 블록으로 출력한다. 업무 서비스는 구현체를 모른다(DIP).

2. 근거

SRS 없음 (엑셀 F열 · 소스만으로 분석). 아래는 요구사항정의서 F열(상세내용) 원문이다.

```
[개요]
결재 · 공지 알림 메일을 MailSender 인터페이스 뒤에서 발송하여, 실제 발송 수단을 바꿔도 업무 코드가 바뀌지 않게 한다.

[상세 기능 요구사항]
1) MailSender.send(받는 주소, 제목, 본문) 하나로 발송한다.
2) 제목 형식은 '[{태그}] {문서 제목}' 이다. 태그: 결재요청 · 결재승인 · 결재반려 · 긴급공지.
3) 본문 형식은 '안녕하세요 {이름}님,\n{안내 문구}\n제목: {문서 제목}' 이며 필요 시 줄을 덧붙인다.
4) 현재 구현체 ConsoleMailSender 는 콘솔에 '=== MAIL ===' 블록으로 출력한다.
5) 받는 사람이 DB 에 없으면 발송하지 않는다.

[입력 / 출력]
- 입력: 받는 주소, 제목, 본문
- 출력: 메일 발송(현재 콘솔 출력)

[제약 조건 / 비기능 요구사항]
- 제목 · 본문 문자열을 한 글자도 바꾸지 않는다(ApprovalNotificationOutputTest 가 출력 비교).
- SMTP 등 실제 발송 구현체를 추가할 때 업무 서비스 코드를 고치지 않는다.

[검증 기준]
- 결재 상신 시 콘솔 출력의 TO 가 결재자 이메일, SUBJECT 가 '[결재요청] {제목}' 이다.
- MailSender 를 대체 구현으로 바꿔 끼워도 기존 테스트가 통과한다.

[작업 대상]
legacy-portal (D:\legacy-portal) Spring Boot 3.2 / Java 21
src\main\java\kr\hull\portal\common (MailSender · ConsoleMailSender · MailFormat)
src\main\java\kr\hull\portal\approval\ApprovalNotifier.java

[Worker Settings]
- MainWorker          : claude-main
```

```
- SubWorker           : codex-critic
- MainWorker-SubAgent : backend-expert
```

3. 분석 결과

3.1 추적표

항목	요구사항(F열 문장)	SRS 근거	소스 위치	판정	비고
상세1	MailSender.send(받는 주소, 제목, 본문) 하나로 발송한다.	SRS 없음	common/MailSender.java:11-13, approval/ApprovalNotifier.java:46, 55, 64, notice/NoticeService.java:100	충족	메서드 1개 void send(String to, String subject, String body). 업무 코드의 발송 호출은 모두 이 메서드.
상세2	제목 형식은 '[{태그}] {문서 제목}' 이다. 태그: 결재요청 · 결재승인 · 결재반려 · 긴급공지.	SRS 없음	common/MailFormat.java:19-21, approval/ApprovalNotifier.java:46, 55, 64, notice/NoticeService.java:100	충족	"[" + tag + "] " + title. 태그 4종 모두 사용처 확인.
상세3	본문 형식은 '안녕하세요 {이름}님,₩n{안내 문구}₩n제목: {문서 제목}' 이며 필요 시 줄을 덧붙인다.	SRS 없음	common/MailFormat.java:24-26, approval/ApprovalNotifier.java:72-84, notice/NoticeService.java:101	충족	덧붙임 줄: 상신 \n기안자ID: {id}, 반려 \n사유: {reason}. 승인·긴급공지는 덧붙임 없음.
상세4	현재 구현체 ConsoleMailSender는 콘솔에 '=== MAIL ===' 블록으로 출력한다.	SRS 없음	common/ConsoleMailSender.java:15-25	충족	=== MAIL === / TO: / SUBJECT: / 본문 / ========== 5줄. 유일한 MailSender 빈(@Component).
상세5	받는 사람이 DB 에 없으면 발송하지 않는다.	SRS 없음	approval/ApprovalNotifier.java:42-45, 51-54, 60-63, 68-70, notice/NoticeService.java:99	충족	결재: findById(...).orElse(null) → null 이면 return. 긴급공지: userRepository.findAll() 대상만 발송. 수신자 id 가 null 이면 findById(null) 예외(레거시 보존).
입력	입력: 받는 주소, 제목, 본문	SRS 없음	common/MailSender.java:13	충족	
출력	출력: 메일 발송(현재 콘솔 출력)	SRS 없음	common/ConsoleMailSender.java:19-25	충족	실제 SMTP 발송 없음.

항목	요구사항(F열 문장)	SRS 근거	소스 위치	판정	비고
제약 1	제목 · 본문 문자열을 한 글자도 바꾸지 않는다 (ApprovalNotificationOutputTest 가 출력 비교).	SRS 없음	`common/MailFormat.java:19-26`, `approval/ApprovalNotifier.java:72-84`	**충족**	`ApprovalNotificationOutputTest`(결재 3종) · `NoticeServiceCharacterizationTest`(긴급공지)가 전체 블록을 문자열 비교.
제약 2	SMTP 등 실제 발송 구현체를 추가할 때 업무 서비스 코드를 고치지 않는다.	SRS 없음	`approval/ApprovalNotifier.java:22-27`, `notice/NoticeService.java:43, 50-63`	**충족**	두 곳 모두 `MailSender` 인터페이스를 생성자 주입. `ApprovalServiceMailUnitTest` 가 서비스 무수정으로 `FakeMailSender` 주입. 다만 두 번째 `@Component` 구현체를 추가하면 빈 중복이 되므로 `@Primary`/`@Profile`/`@ConditionalOnProperty` 등 설정 측 선택이 필요(서비스 코드는 무수정).

3.2 판정 요약

충족 **9** / 부분 충족 **0** / 미충족 **0** / 불일치 **0** / 확인 불가 **0**

4. 구성 요소 설계

클래스	책임	주요 메서드 시그니처 (현행 유지)
`common.MailSender` (interface)	발송 추상화	`void send(String to, String subject, String body)`
`common.ConsoleMailSender` (@Component)	현재 구현체 - 콘솔 출력	`public void send(String to, String subject, String body)`
`common.MailFormat` (final, 정적)	제목·본문 양식 한 곳	`public static String subject(String tag, String title)` `public static String body(String recipientName, String message, String title)`

`approval.ApprovalNotifier` (@Component)	결재 알림 수신자·말머리·안내문 결정	`public void notifySubmitted(Approval approval)`
		`public void notifyApproved(Approval approval)`
		`public void notifyRejected(Approval approval, String reason)`

`notice.NoticeService`	긴급공지 게시 시 전체 사용자에게 발송	`private void notifyAllMembersOfEmergency(Notice notice)`
(테스트) `common.FakeMailSender`	대체 구현 - 호출 내용만 기록	`List<SentMail> getSentMails()`

참고: `common/SmtpMailSender.java` 는 `MailSender` 를 구현하지 않고 어디서도 참조되지 않는 레거시 클래스(git 미추적, 작업 트리에만 존재)로, 이름과 달리 콘솔에 같은 블록을 출력한다.

5. 인터페이스 설계

5.1 Java 인터페이스

파라미터	타입	설명	예시
to	String	받는 주소 (users.email)	park@hull.kr
subject	String	[{태그}] {문서 제목}	[결재요청] 노트북 구매
body	String	안녕하세요 {이름}님,\n{안내}\n제목: {제목} + 덧붙임	안녕하세요 박팀장님,₩n결재 요청이 도착했습니다.₩n제목: 노트북 구매₩n기안자ID: 1

5.2 알림 종류

계기	수신자	태그	안내 문구	덧붙임 줄
결재 상신 (0→1)	결재자	결재요청	결재 요청이 도착했습니다.	₩n기안자ID: {drafterId}
결재 승인 (1→2)	기안자	결재승인	결재가 승인되었습니다.	없음
결재 반려 (1→3)	기안자	결재반려	결재가 반려되었습니다.	₩n사유: {reason}
긴급공지 게시	전체 사용자(findAll 순)	긴급공지	긴급 공지가 게시되었습니다.	없음
결재 취소 · 일정 · 일반 공지	메일 없음			

5.3 콘솔 출력 형식 (ConsoleMailSender)

```
=== MAIL ===
TO: park@hull.kr
SUBJECT: [결재요청] 노트북 구매
안녕하세요 박팀장님,
결재 요청이 도착했습니다.
제목: 노트북 구매
기안자ID: 1
============
```

오류 동작: 수신자 미존재 → 발송 생략(예외 없음). 수신자 id null → `findById(null)` 예외(보존). 발송 실패 처리·재시도 개념 없음.

6. 데이터 설계

메일 자체는 저장하지 않는다. 읽기 대상만 있다.

테이블	사용 컬럼	용도
`users`	id, name, email	수신자 조회·인사말 이름·받는 주소
`approval`	title, drafter_id, approver_id	제목·수신자 결정·기안자ID 줄
`notice`	title, category(2=긴급)	제목·긴급 여부

상태 전이: 메일은 결재 상태 0→1, 1→2, 1→3 과 공지 0→1(긴급)이 **실제로 일어난 뒤**에만 발송된다(DB 저장값은 정수 그대로).

7. 처리 흐름

순서	단계	결재 상신 예	소스
1	상태 변경 가드	`approval.submit(...)` false 면 종료 (메일 없음)	`ApprovalService.java:140-142`
2	저장	`approvalRepository.save`	`ApprovalService.java:143`
3	수신자 조회	`findRecipient(approverId)` - 없으면 종료	`ApprovalNotifier.java:42-45, 68-70`
4	양식 조립	`MailFormat.subject` · `submittedBody`	`ApprovalNotifier.java:46, 72-75`
5	발송	`mailSender.send(email, subject, body)`	`ApprovalNotifier.java:46`
6	감사로그	`APPROVAL SUBMIT` (메일 다음)	`ApprovalService.java:145`

긴급공지: 게시 가드 → 저장 → 전체 사용자 루프 발송 → `NOTICE PUBLISH` 감사로그 (`NoticeService.java:87-94, 98-103`).

8. 예외 · 경계 처리

상황	현재 동작	레거시 보존 여부
결재자/기안자가 DB 에 없음	메일 생략, 감사로그는 남음	보존
수신자 id null	`findById(null)` 예외	보존 (개선 대상)
반려 사유 null	본문에 사유: `null`	보존 (소스상 문자열 연결 결과, 테스트 미고정)
사용자 email null	`TO: null` 출력	보존 (소스상 문자열 연결 결과, 테스트 미고정)
조건 불만족으로 무시된 처리	메일 없음	보존

9. 검증 설계

검증 기준 (F열)	고정 테스트
결재 상신 시 콘솔 출력의 TO 가 결재자 이메일, SUBJECT 가 '[결재요청] {제목}' 이다.	`ApprovalNotificationOutputTest#상신하면_결재자에게_결재요청_메일과_감사로그가_나간다`
MailSender 를 대체 구현으로 바꿔 끼워도 기존 테스트가 통과한다.	`ApprovalServiceMailUnitTest` 7개 (FakeMailSender 주입), 예: `#상신하면_결재자에게_결재요청_메일을_한통_보낸다`, `#가짜_발송기를_주입하면_콘솔로_실제_메일이_나가지_않는다`

보조: 미존재 수신자 `RequirementAcceptanceTest#받는_사람이_DB에_없으면_메일을_보내지_않는다`, 승인·반려 `ApprovalNotificationOutputTest#승인하면_기안자에게_결재승인_메일과_감사로그가_나간다` · `#반려하면_기안자에게_사유가_담긴_결재반려_메일과_감사로그가_나간다`, 긴급공지 `NoticeServiceCharacterizationTest#긴급_공지를_게시하면_전체_사용자에게_긴급공지_메일이_한통씩_나간다`.

추가 테스트 제안

제안 메서드명	내용 (변수는 영어)
가짜_발송기를_주입하면_긴급공지_메일이_사용자_수만큼_기록된다()	`FakeMailSender fakeMailSender` 로 `NoticeService` 조립 → 긴급 공지 게시 → `getSentMails()` 크기 = 사용자 수, subject "[긴급공지] ..." (공지 쪽 대체 구현 검증 보강)

제안 메서드명	내용 (변수는 영어)
승인할_때_기안자가_DB에_없으면_메일을_보내지_않는다()	drafterId 를 없는 id 로 둔 결재를 상신·승인 → sentMails 에 결재승인 없음

10. 검토 의견 · 후속 과제

구분	내용
보존 대상	MailSender.send(to, subject, body) 시그니처, 제목·본문·덧붙임 줄·콘솔 블록 문자열(1글자도), 수신자 결정 규칙, 수신자 미존재 시 생략, 수신자 id null 예외, 저장 → 메일 → 감사로그 순서.
개선 대상	(1) 실제 SMTP 구현체 추가 시 @ConditionalOnProperty("portal.mail.mode") 또는 @Profile 로 빈 선택 - 서비스 무수정. (2) 미사용 레거시 common/SmtpMailSender.java(MailSender 미구현·콘솔 출력)는 실제 SMTP 구현과 이름이 충돌하므로 정리 대상(삭제 또는 이름 변경 - 사용자 확인 후). (3) 발송 실패 시 업무 트랜잭션 영향·재시도 정책 미정의. (4) 수신자 id null 방어는 동작 변경이므로 재설계 단계.
연계 요구사항	REQ-F-002(상신), REQ-F-003(승인), REQ-F-004(반려), REQ-F-010(긴급공지), REQ-F-013(사용자 email), REQ-IF-002(감사로그 순서)